

基於 ESP32 之輕量化多模態物聯網入侵偵測系統：從國科會計畫書轉向執行階段之技術深度報告與學術指引

物聯網邊緣運算環境下的安全防護範式與多模態偵測之理論基礎

在當前物聯網(IoT)生態系統中，設備的安全監測正經歷一場從雲端中心化偵測轉向邊緣端自主偵測的範式轉移¹。這種轉變的主要驅動力來自於對即時性、頻寬成本以及隱私保護的極致追求。傳統的入侵偵測系統(IDS)往往依賴於將大量流量數據回傳至高性能伺服器進行深度包檢測(DPI)，然而對於資源極度受限的端點設備如 ESP32 或 ESP8266 而言，這種模式不僅會產生巨大的通訊延遲，更可能因頻寬受限而丟失關鍵的攻擊特徵¹。

本研究計畫所提出的「多模態物聯網入侵偵測系統」旨在解決單一偵測維度的侷限性。現有的物聯網安全研究大多呈現碎片化特徵：一類研究專注於網路入侵偵測系統(NIDS)，透過分析封包元數據(Metadata)或物理層特徵如訊號強度指示(RSSI)來識別 Wi-Fi 攻擊；另一類則專注於主機入侵偵測系統(HIDS)，藉由監控系統日誌(Syslog)來發現軟體執行邏輯的異常¹。然而，單一 NIDS 在面對加密流量(如 TLS/SSL)時往往會出現「偵測盲區」，而單一 HIDS 則難以感知發生在物理層或連線層的網路擾動，如去識別化攻擊(Deauthentication Attack)或惡意雙胞胎(Evil Twin)攻擊¹。

透過結合裝置內部的系統日誌(Syslog)與網路流量特徵，多模態融合技術能夠建立起一種「跨時域與跨空間」的關聯檢驗機制。例如，當 NIDS 偵測到瞬間湧入的大量管理幀(Management Frames)時，若系統內部的堆積記憶體(Free Heap)同時出現顯著波動或異常重啟，這種跨模態的因果鏈結能將誤報率(False Positive Rate)與漏報率(False Negative Rate)大幅降低¹。在資源受限的環境下，如何將複雜的融合模型輕量化並部署於僅具備 520KB RAM 的微處理器中，是本計畫從提案階段轉向執行階段的核心技術難點¹。

關鍵文獻深度解析：從 Moharam 2025、Lo 2023 到 Lin 2022 的技術啟示

在計畫啟動初期，重點研讀領域內的前沿文獻是建立執行基準的必要步驟。根據計畫書之規劃，以下三篇研究為當前技術路徑的重要參考點。

Moharam 等人之 Wi-Fi 攻擊即時偵測研究

Moharam 等人在 2025 年發表的研究，成功在 ESP8266 (NodeMCU) 平台上實現了基於混合深度學習模型的 Wi-Fi 攻擊偵測²。該研究的重要性在於證明了在極低成本的嵌入式設備上進行即時封包嗅探(Sniffing)與特徵提取的可行性。其採用的特徵集包含 RSSI、SNR 以及去識別化封包計

數，並透過長短期記憶網路(LSTM)與閘道器循環單元(GRU)結合邏輯回歸(LR)進行分析⁴。這為本計畫提供了 NIDS 子系統的基準指標，特別是在利用物理層數據監測無線攻擊方面具有高度參考價值⁴。

Lo 等人之 AIoT 設備輕量化模型優化技術

Lo、Cheng 與 Kaliski 在 2023 年的研究專注於如何將強大的機器學習模型優化並移植至低階 AIoT 設備³。由於 scikit-learn 等主流庫在嵌入式環境中會因依賴性過高而導致記憶體耗盡，Lo 等人提出了「零依賴性(Zero-dependency)」的部署原則。該研究利用模型編譯轉換技術(如 m2cgen 或 emlearn)，將訓練好的隨機森林(Random Forest)或多層感知器(MLP)轉換為純 C 語言頭文件，從而移除對大型函式庫的依賴³。這為本計畫解決 Q3(資源受限型模型產出)提供了具體的實踐方法論¹。

Lin 等人之多數據源融合入侵偵測

Lin 等人在 2022 年的研究深入比較了系統日誌、封包流與主機統計數據在不同攻擊階段的偵測效能⁵。研究結果顯示，主機統計數據(Host Statistics)在偵測影響階段(Impact Stage)表現最佳，而系統日誌在初始存取階段(Initial Access)具有顯著優勢⁵。這項發現強化了本計畫採取多模態融合的動機，即透過數據互補來覆蓋完整的攻擊生命週期¹。

下表總結了這些關鍵文獻與本計畫執行重點的關聯性¹：

文獻來源	核心技術貢獻	對本計畫之具體指引
Moharam (2025)	物理層特徵 (RSSI, SNR) 的即時嗅探與解析	NIDS 子系統的特徵選取與回調函數實作基準
Lo (2023)	零依賴性模型轉譯與隱藏節點剪枝優化	解決 ESP32 RAM 限制，達成模型嵌入式部署
Lin (2022)	多數據源 (Packet+Log) 的分階段效能評估	多模態融合策略與攻擊場景設計之理論支持
DeepLog (2017)	利用 LSTM 捕捉系統日誌執行序列	HIDS 子系統針對異常重啟與資源波動之建模參考

硬體平台特性與資源限制下的架構設計：ESP32 與微控制器性能分析

本計畫選用 ESP32 作為核心開發平台，主因在於其具備雙核(Dual-core)處理能力。在執行入侵

偵測任務時，一個核心可以被專用於 Wi-Fi 驅動程序與混雜模式下的封包接收，而另一個核心則負責計算密集的特徵對齊、模型推理以及 Syslog 格式轉換¹。

ESP32 與其前代 ESP8266 的性能差異直接決定了資料處理的緩衝區深度 (Buffer Depth)。ESP32 擁有約 520KB 的 SRAM，但在實際開發中，當 Wi-Fi 堆疊與 MicroPython 韌體載入後，可用於應用層的靜態記憶體往往剩下不到 200KB¹。這意味著在設計「非同步緩存機制」時，必須精確計算每個特徵向量的字節長度。

下表詳列了兩款主流微控制器的硬體規格對比，這對於計畫執行中的 Q1 (低侵入式監控可行性) 評估至關重要¹：

硬體參數	ESP32	ESP8266 (NodeMCU)
CPU 核心	雙核 Xtensa® LX6 (160~240 MHz)	單核 Tensilica L106 (80~160 MHz)
SRAM 總量	520 KB	< 50 KB (實質可用約 32 KB)
Flash 儲存	4 MB (可擴充)	4 MB
Wi-Fi 吞吐量	支持 802.11 n (高達 150 Mbps)	支持 802.11 b/g/n (低速)
安全硬體加速	AES, SHA2, RSA, ECC, RNG	僅基礎硬體加速

在執行初期，研究者必須注意 ESP32 的內部分區表 (Partition Table) 配置。由於本計畫涉及模型存儲與日誌臨時緩衝，建議採用「Huge APP」分區方案或外掛 SPIFFS/LittleFS 文件系統，以確保有足夠的 Flash 空間存放轉換後的 C 語言模型頭文件¹³。

網路入侵偵測子系統 (NIDS)：混雜模式、封包過濾與物理層特徵提取機制

NIDS 的核心在於對 Wi-Fi 通訊的底層觀測。透過調用 ESP-IDF 中的 `esp_wifi_set_promiscuous(true)`，ESP32 可以繞過標準的過濾機制，接收所有在同一頻道 (Channel) 內傳輸的無線訊號¹⁰。然而，全頻道的封包擷取會對 CPU 產生巨大壓力，因此必須實作硬體過濾器與回調函數的優化。

封包過濾與回調函數優化

在實務執行中，應使用 `esp_wifi_set_promiscuous_filter()` 來指定感興趣的封包類型¹¹。本計畫重

點關注管理幀 (Management Frames, 如 Probe Request, Deauthentication) 與數據幀 (Data Frames)¹¹。研究指出, 在混雜模式下, 回調函數 `wifi_promiscuous_cb_t` 會直接在 Wi-Fi 驅動任務中被調用, 若在此函數中執行過重的邏輯 (如大量字串列印或複雜運算), 會導致網路協定棧崩潰¹¹。

解決方案是採用「生產者-消費者模式」。回調函數僅負責將封包的中繼數據 (Metadata) 寫入一個快速的環形緩衝區 (Ring Buffer), 而模型推理任務則定期從緩衝區中讀取數據進行處理¹⁷。

特徵提取: RSSI 與 SNR 的動態校正

本計畫的一個核心研究問題 (Q4) 是物理環境干擾下的穩定性¹。RSSI 是衡量攻擊的重要指標, 但其值會隨設備距離、障礙物甚至空氣濕度而波動。參考 Moharam 的方法, 特徵集應包含:

- 1. **RSSI (Received Signal Strength Indicator):** 訊號強度, 以 dBm 為單位。
- 2. **SNR (Signal-to-Noise Ratio):** 訊號雜訊比, 反映通訊品質。
- 3. **Packet Count:** 特定時間窗口內的總幀數。
- 4. **Deauth Count:** 去識別化封包的頻率。

為了消除噪訊, 應在 NIDS 端實作動態基準值校正。計算公式如下:

$$RSSI_{adj} = RSSI_{raw} - \alpha \cdot \overline{RSSI}_{normal}$$

其中 α 為環境補償因子, 這能確保模型在不同的部署環境下具有一致的特徵分布¹。

系統日誌監控子系統 (HIDS): RFC 5424 標準化與主機行為特徵化

相對於 NIDS 關注「外部威脅」, HIDS 則監控設備的「內部健康」。計畫中明確要求對齊 RFC 5424 標準, 這是一項極具工程價值的決定¹。RFC 5424 提供了一種結構化的日誌格式, 包含版本號、精確的時間戳記、主機名、應用程序名、進程 ID 以及結構化數據 (Structured Data) 段¹⁹。

RFC 5424 與傳統 RFC 3164 之對比

下表展現了標準化對自動化分析的重要性¹:

屬性	RFC 3164 (BSD Syslog)	RFC 5424 (IETF Syslog)
時間戳記精確度	秒 (無年份、無毫秒)	毫秒/微秒 (ISO 8601 格式)
字符編碼	僅限 ASCII	支援 UTF-8 (含 BOM)

擴展性	低 (依賴字串解析)	高 (支持 Structured Data 選項)
時間對齊可靠性	容易受主機時鐘影響	支持 UTC 偏移量與精確標註

在執行過程中，ESP32 應實作一個輕量級的編碼器，將 Free_Heap、Task_Status、Reset_Reason 等關鍵指標封裝進 RFC 5424 的結構化數據部分。例如，SDID (Structured Data ID) 可以定義為 ``¹⁹。這樣的設計使得後端的分析模型 (如 XGBoost) 可以直接解析出數值特徵，而無需進行耗時的正規表達式 (Regex) 字串匹配 ¹。

日誌異步緩存與發送機制

由於 Wi-Fi 連線在攻擊期間 (如 DDoS 或 Deauth 攻擊下) 可能不穩定，HIDS 必須具備斷網暫存功能。建議實作一個基於 Flash 的循環日誌緩衝區 ¹⁴。利用 ESP32 的 Wear Leveling 組件，可以確保在 Flash 上的頻繁讀寫不會縮短硬體壽命 ¹⁴。當網路恢復後，系統再將積壓的 RFC 5424 日誌異步回傳至邊緣閘道器上的 rsyslog 服務 ¹。

多模態數據融合之技術瓶頸：高精度時間同步與非同步緩存對齊演算法

本計畫的核心創新點在於「多模態融合」，而融合的成敗取決於 Q5 (資料對齊與標準化技術瓶頸) 的解決 ¹。NIDS 捕捉到的毫秒級封包特徵必須與 HIDS 記錄的系統狀態精確匹配。然而，微控制器在執行混雜模式監聽與日誌生成時，往往會產生顯著的處理延遲 (Latency Jitter)。

時間同步：NTP 與硬體時間戳記的應用

傳統的 NTP (Network Time Protocol) 在網際網路上能提供 10ms 至 100ms 的精度，但在區域網內透過精細優化可達 1ms 級別 ²⁵。對於本計畫而言，單純的軟體 NTP 可能不足以應對高速網路攻擊的取樣需求。解決方案是利用 Wi-Fi 硬體計時器 ²⁶：

- Promiscuous RX Timestamp:** ESP32 的 Wi-Fi 物理層會在封包抵達的瞬間打上一個基於 1μs 解析度的硬體時間戳記 wifi_pkt_rx_ctrl_t.timestamp ¹⁶。
- 時鐘飄移補償演算法:** 由於硬體計時器與系統牆鐘 (Wall Clock) 存在漂移，應實作線性回歸模型 (Linear Regression) 來校正。

$$T_{global} = \beta_0 + \beta_1 \cdot T_{local_tick}$$

透過定期獲取 NTP 同步點，計算出最優的 β_0 (偏移量) 與 β_1 (速率係數)，從而將本地硬體時間轉換為精確的全局時間 ²⁶。

滑動視窗融合演算法

為了將異構數據(異步產生的流量特徵與日誌事件)對齊，應實作「基於時間戳記的滑動視窗機制」。將時間維度切分為 100ms 的細粒度槽位¹。在每個槽位內，系統聚合封包特徵(如平均 RSSI、總流量)與系統日誌快照(如當前 Heap 餘量)。這種「特徵級融合 (Feature-level Fusion)」能保留數據間的深度關聯，優於單純的決策級融合。其邏輯如下表所示¹：

時間窗口 (T)	NIDS 特徵 (Packet Counts)	HIDS 特徵 (Free Heap)	物理層特徵 (Mean RSSI)	標籤 (Label)
T + 100ms	12	124,560 B	-65 dBm	Normal
T + 200ms	450 (突發)	110,200 B (下降)	-88 dBm (干擾)	Attack (Deauth)
T + 300ms	820 (飽和)	105,400 B (持續下降)	-92 dBm	Attack (DoS)

輕量化模型優化與轉譯技術：從 Python 訓練到 C 語言嵌入式部署

本計畫的 Q3 旨在解決如何產出能嵌入 ESP32 僅 520KB 記憶體的模式¹。這需要一個「訓練與部署分離」的流程：利用高效能 PC 使用大樣本數據集(如 CICIoT2023 或自行採集的數據)訓練模型，再將其轉譯為輕量化 C 代碼。

模型轉譯器：m2cgen 與 emlearn 的應用

在執行階段，應重點測試以下兩款開源工具⁷：

- 1. **m2cgen (Model 2 Code Generator)**: 這是將 scikit-learn 模型轉化為「零依賴」C 代碼的最佳選擇。它會將決策樹的 if-else 邏輯直接展開為嵌入式代碼。缺點是若樹的深度太深，生成的代碼文件會過大(Flash 占用增加)³¹。
- 2. **emlearn**: 專為微控制器設計的機器學習庫，其優勢在於支持固定點數 (Fixed-point) 運算，能避開硬體浮點運算器 (FPU) 的限制，提升推理速度。且 emlearn 提供 MicroPython 綁定，適合本計畫快速原型開發的需求³²。

知識蒸餾與剪枝策略

為了進一步優化，計畫建議採用「知識蒸餾 (Knowledge Distillation)」技術¹。使用一個大型的隨機森林(例如 100 棵樹)作為教師模型，去訓練一個小型的學生模型(例如 10 棵樹)。根據 Lo 等人的

實驗, 透過適度的剪枝(Pruning)與節點縮減, 可以在僅損失 <0.5% 準確度的情況下, 將 RAM 占用從數十 MB 降至 20KB 以內³。

下表展示了優化前後的模型效能預期對比⁶:

模型性能指標	標準 Python 模型 (PC)	優化後 C 模型 (ESP32)
推理延遲 (Inference Latency)	~200 ms	< 1 ms
記憶體占用 (RAM Footprint)	~50 MB	< 20 KB
依賴項 (Dependencies)	numpy, pandas, sklearn	無 (Pure C Header)
偵測準確率 (Accuracy)	98.2%	97.5% (微幅損失)

計畫執行之實務路徑: 每週進度管理與研究產出之結構化建議

針對每週一次的教授會議, 執行者需要將研究階段 (Architecture -> Data -> Model -> Validation) 拆解為可交付的、具備數據支持的小型里程碑。這不僅能維持研究進度, 更能透過週會反饋修正技術路線。

第一個月: 基礎建設與資料採集管線 (每週進度安排)

- 第 1 週: 硬體開發環境與 **Hello World**。完成 ESP-IDF 與 MicroPython 雙環境搭建。展示能同時運行的 Wi-Fi 掃描與內置感測器數據讀取。
- 第 2 週: 混雜模式數據解析。實作 esp_wifi_set_promiscuous_rx_cb, 展示能實時在 Serial Console 打印出周圍設備的 MAC 位址、封包類型與 RSSI。
- 第 3 週: **RFC 5424** 日誌編碼器。完成 C 語言版本的 RFC 5424 格式化函數。驗證產出的日誌能被外部 syslog 伺服器正確解析出 Structured Data 欄位。
- 第 4 週: 異步環形緩存實作。完成跨核心數據傳遞邏輯。測試在 1000 封包/秒的極限壓力下, 系統是否會發生堆積溢位 (Heap Overflow)。

第二個月: 攻擊場景模擬與特徵化 (每週進度安排)

- 第 5 週: 攻擊環境搭建與 **Kali Linux** 整合。在隔離的 Sandboxed Wi-Fi 環境下, 實作自動化的去識別化攻擊 (Deauth Attack) 腳本。
- 第 6 週: 異常數據特徵分布分析。繪製正常與攻擊狀態下的 RSSI 分佈圖 (Histogram)。計算兩者之間的 Jensen-Shannon 距離, 證明特徵具備可區分性。
- 第 7 週: 系統狀態特徵化。模擬 TCP SYN Flood 攻擊。監測並導出攻擊期間的 Free_Heap 與

Context_Switch_Rate 變化。

- 第 8 週：多模態資料集初步產出。完成第一版「流量+日誌」對齊資料集。報告資料集的樣本均衡度，並進行初步的關聯統計(Correlation Analysis)。

第三個月：模型優化與推理引擎開發(每週進度安排)

- 第 9 週：**Baseline** 模型訓練。在 PC 端使用 XGBoost 訓練初始模型。展示初步的 Confusion Matrix 與 ROC 曲線。
- 第 10 週：特徵重要性排名。向教授報告哪些特徵(如 RSSI 或是重啟原因)對模型判斷最為關鍵。
- 第 11 週：模型轉譯與 **m2cgen** 測試。將訓練好的模型轉換為 model.h。比較不同樹深度對生成代碼行數(Line Counts)的影響。
- 第 12 週：**emlearn** 與 **MicroPython** 整合。在 ESP32 上調用 C 模組執行推理。報告單次推理的耗時與 CPU 頻率的關係。

第四個月：系統驗證與效能評估(每週進度安排)

- 第 13 週：即時偵測準確度驗證。進行 Live Demo 級別的測試：當 Kali Linux 發動攻擊，ESP32 需在 1 秒內在 OLED 或 Serial 上報出正確的攻擊類型。
- 第 14 週：動態環境校正測試。變更設備位置(移動設備 5-10 米)，測試 Q4 提出的自適應校正方法是否能有效抑制誤報。
- 第 15 週：防禦機制與 **SDN** 聯動。實作當偵測到威脅時，ESP32 自動發送特定 UDP 封包給路由器的防火牆，執行攔截。
- 第 16 週：結案報告數據彙整。整理所有圖表。包含資源占用率、偵測延遲與偵測種類的總結。

結論：邁向更具韌性的物聯網邊緣防禦生態

本研究計畫的執行，標誌著物聯網安全防禦從「被動上傳」轉向「主動邊緣識別」的重要跨越。透過結合 ESP32 的多核算力、標準化 RFC 5424 日誌協議以及零依賴性的模型優化技術，我們能夠在不增加雲端負擔的前提下，為物聯網節點賦予神經網路般的防禦能力¹。

在執行過程中，重點應放在「數據對齊的精確性」與「模型推論的極簡性」。計畫所擬定的週會進度，不僅是為了產出學術論文，更是為了建立一套可重現的資安資料採集管線(Data Pipeline)¹。未來的擴展方向可考慮引入可解釋性 AI (XAI) 技術，當系統報警時，能具體指出是因「RSSI 異常抖動」還是「Heap 被惡意占用」所引起，這將大幅提升維運工程師的響應效率¹。

計畫的最終產出——一套能夠部署在 ESP32 上的多模態偵測框架，將為工業 4.0 或是智慧家居環境提供低成本且高效能的安全解決方案。這不僅解決了當前的網路偵測盲區，更為未來建立大規模自組織安全網路(Self-Organizing Security Networks)奠定了堅實的實證基礎。

引用的著作

1. 研究計畫摘要表(C802).pdf

2. Real-time detection of Wi-Fi attacks using hybrid deep learning models on NodeMCU, 檢索日期: 2月 22, 2026, <https://communities.springernature.com/posts/real-time-detection-of-wi-fi-attacks-using-hybrid-deep-learning-models-on-nodemcu>
3. Optimization of Lightweight Malware Detection Models For IoT Devices - arXiv, 檢索日期: 2月 22, 2026, <https://arxiv.org/html/2404.04567v1>
4. (PDF) Real-time detection of Wi-Fi attacks using hybrid deep ..., 檢索日期: 2月 22, 2026, https://www.researchgate.net/publication/395525994_Real-time_detection_of_Wi-Fi_attacks_using_hybrid_deep_learning_models_on_NodeMCU
5. Multi-datasource machine learning in intrusion detection: Packet ..., 檢索日期: 2月 22, 2026, https://www.researchgate.net/publication/362967369_Multi-datasource_machine_learning_in_intrusion_detection_Packet_flows_system_logs_and_host_statistics
6. A Comparative Analysis of Single and Multi-View Deep Learning for Cybersecurity Anomaly Detection - IEEE Xplore, 檢索日期: 2月 22, 2026, <https://ieeexplore.ieee.org/iel8/6287639/10820123/10975758.pdf>
7. Optimization of Lightweight Malware Detection Models For ... - arXiv, 檢索日期: 2月 22, 2026, <https://arxiv.org/abs/2404.04567>
8. Intrusion Detection Research Research Articles - Page 1 | R Discovery, 檢索日期: 2月 22, 2026, https://discovery.researcher.life/topic/research-in-intrusion-detection/24288412?page=1&topic_name=Research%20In%20Intrusion%20Detection
9. Van-Linh Nguyen - Google Scholar, 檢索日期: 2月 22, 2026, <https://scholar.google.com/citations?user=M7iGWiEAAAAJ&hl=en>
10. ESP32 - WiFi sniffer - Łukasz Podkalicki, 檢索日期: 2月 22, 2026, <https://blog.podkalicki.pl/esp32-wifi-sniffer/>
11. Wi-Fi Modes - ESP32 - — ESP-IDF Programming Guide latest documentation, 檢索日期: 2月 22, 2026, <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/wifi-driver/wifi-modes.html>
12. functionality specific to the ESP32 — MicroPython latest documentation, 檢索日期: 2月 22, 2026, <https://docs.micropython.org/en/latest/library/esp32.html>
13. Recurring Updates to ESP32 Time on LED Clock Using NTP Server - Arduino Forum, 檢索日期: 2月 22, 2026, <https://forum.arduino.cc/t/recurring-updates-to-esp32-time-on-led-clock-using-ntp-server/1239699>
14. Best Practice to Implement a Circular Logger on Internal Flash - ESP32 Forum, 檢索日期: 2月 22, 2026, <https://esp32.com/viewtopic.php?t=29055>
15. ESP32 WiFi MAC Scanner/Sniffer (promiscuous). - Hackster.io, 檢索日期: 2月 22, 2026, <https://www.hackster.io/p99will/esp32-wifi-mac-scanner-sniffer-promiscuous-4c12f4>
16. Timestamp accuracy in wifi_promiscuous_pkt_type_t - ESP32 Forum, 檢索日期: 2月 22, 2026, <https://esp32.com/viewtopic.php?t=6051>

17. An asynchronous lock free ring buffer for logging, 檢索日期: 2月 22, 2026, <https://steven-giesel.com/blogPost/11f0ded8-7119-4cfc-b7cf-317ff73fb671>
18. Circular buffer - Wikipedia, 檢索日期: 2月 22, 2026, https://en.wikipedia.org/wiki/Circular_buffer
19. How to Parse Syslog RFC 5424 and RFC 3164 Messages with the Syslog Receiver in the Collector - OneUptime, 檢索日期: 2月 22, 2026, <https://oneuptime.com/blog/post/2026-02-06-syslog-rfc5424-rfc3164-receiver/view>
20. RFC 5424 - The Syslog Protocol - IETF Datatracker, 檢索日期: 2月 22, 2026, <https://datatracker.ietf.org/doc/html/rfc5424>
21. arcao/Syslog: An Arduino library for logging to Syslog server in IETF format (RFC 5424) and BSD format (RFC 3164) - GitHub, 檢索日期: 2月 22, 2026, <https://github.com/arcao/Syslog>
22. rfc5424-logging-handler 1.0.1 - PyPI, 檢索日期: 2月 22, 2026, <https://pypi.org/project/rfc5424-logging-handler/1.0.1/>
23. An RFC5424-Compliant Syslog Handler for the Python Logging Framework - GitHub, 檢索日期: 2月 22, 2026, <https://github.com/jobec/rfc5424-logging-handler>
24. PicoSyslog: A tiny ESP8266 & ESP32 library for sending logs to a Linux Syslog server, 檢索日期: 2月 22, 2026, https://www.reddit.com/r/esp32/comments/1jf3p9k/picosyslog_a_tiny_esp8266_esp32_library_for/
25. PTP vs NTP: Key Differences & Use Cases Explained - Syncworks, 檢索日期: 2月 22, 2026, <https://syncworks.com/ptp-vs-ntp-key-differences-use-cases/>
26. High Precision NTP Client for the ESP32 - Lectrobox, 檢索日期: 2月 22, 2026, <https://www.lectrobox.com/projects/esp32-ntp/>
27. Let users access timestamp of reception in promiscuous mode and TX timestamp of sent packets (802.11mc FTM) (IDFGH-8374) · Issue #9843 · espressif/esp-idf - GitHub, 檢索日期: 2月 22, 2026, <https://github.com/espressif/esp-idf/issues/9843>
28. Sliding Window Algorithm: Simple Explanation with Examples and Code - NxtWave, 檢索日期: 2月 22, 2026, <https://www.ccbp.in/blog/articles/sliding-window-algorithm>
29. Sliding Window: Variable Length + Fixed Length - AlgoMap, 檢索日期: 2月 22, 2026, <https://algomap.io/lessons/sliding-window>
30. Mastering the Sliding Window Technique: A Comprehensive Guide | by Nikhil Bajpai, 檢索日期: 2月 22, 2026, <https://medium.com/@nikhil.cse16/mastering-the-sliding-window-technique-a-comprehensive-guide-6bb5e1e86f99>
31. Jane Street: Gradient Boosted C++ - Kaggle, 檢索日期: 2月 22, 2026, <https://www.kaggle.com/code/jtrotman/jane-street-gradient-boosted-c>
32. emlearn - PyPI, 檢索日期: 2月 22, 2026, <https://pypi.org/project/emlearn/>
33. [P] How to easily deploy XGBoost models in C++ production environments - Reddit, 檢索日期: 2月 22, 2026, https://www.reddit.com/r/MachineLearning/comments/c50nrn/p_how_to_easily_deploy_xgboost_models_in_c/

34. (PDF) emlearn-micropython: Machine Learning and Digital Signal Processing for MicroPython - ResearchGate, 檢索日期: 2月 22, 2026,
https://www.researchgate.net/publication/398913486_emlearn-micropython_Machine_Learning_and_Digital_Signal_Processing_for_MicroPython
35. Machine Learning and Digital Signal Processing for MicroPython. Provides convenient and efficient MicroPython modules, and enables MicroPython application developers to run efficient Machine Learning models on microcontroller, without having to touch any C code. - emlearn-micropython documentation - Read the Docs, 檢索日期: 2月 22, 2026,
<https://emlearn-micropython.readthedocs.io/en/latest/source/README.html>
36. Optimization of Lightweight Malware Detection Models for AIoT Devices - ResearchGate, 檢索日期: 2月 22, 2026,
https://www.researchgate.net/publication/381017220_Optimization_of_Lightweight_Malware_Detection_Models_for_AIoT_Devices
37. Professor Kaliski's Site - CV Highlights, 檢索日期: 2月 22, 2026,
<https://sites.google.com/view/rkaliski/home-page/cv-highlights>